Archeon 3D: Architectural Evolution & AI-Reproducible Design

Forked from: Tencent-Hunyuan/Hunyuan3D-2.1

This document details the transformation of the monolithic/script-based Hunyuan3D model into Archeon 3D, a local-first, high-performance 3D generation backend designed for native app integration (Tauri).

---

1. Core Architectural Shift: The "Sidecar" Pattern

Hunyuan3D-2.1 is primarily a research-oriented codebase centered around Gradio or individual inference scripts. Archeon 3D migrates this to a **Sidecar Architecture**.

Key Improvements:

- **Decoupled Frontend**: The GUI is a modern HTML/JS/CSS frontend served by a FastAPI backend, enabling native desktop integration via Tauri.
- **Unified Entry Point**: `archeon_3d.py` handles environment setup, dynamic port allocation, and sub-process lifecycle management.
- **Dynamic Port Allocation**: Uses `find_free_port` to avoid collisions with other local dev servers (e.g., Gradio defaults).

---

2. Robust Backend Management (The Engine Room)

The original repo loads models directly in a script. Archeon 3D introduces two critical abstractions in `hy3dgen/manager.py`:

ModelManager (Resource Isolation)

- **LRU Cache & VRAM Safety**: Automatically offloads models from VRAM when switching between `Normal` (Text-to-3D) and `Multiview` pipelines.
- **Lazy Loading**: Models are registered as loaders and only fetched into memory when the first request hits the queue.
- **Aggressive Garbage Collection**: Explicitly triggers `gc.collect()` and `torch.cuda.empty_cache()` after every generation step.

PriorityRequestManager (Asynchronous Queue)

- **Non-blocking Execution**: Uses `asyncio.PriorityQueue` to handle multiple incoming API requests.
- **Worker Loops**: A configurable number of workers (default: 1 for low-VRAM safety) process jobs sequentially from the queue.
- **Job Cancellation**: Uses `threading.Event` to propagate cancellation signals from the API down to the inference pipeline.

---

3. Polymorphic API Design

The API in `hy3dgen/api/` is built on **Pydantic V2**, ensuring type safety and documentation generation.

Schema Patterns (`schemas.py`):

- **Universal Job Model**: `JobRequest` wraps inputs, constraints (resolution, format), quality presets, and post-processing steps.
- **MeshOps Integration**: A dedicated `MeshOpsRequest` allows for "post-generation" transformations (repair, decimation, auto-texturing).
- **Polymorphic Endpoints**: `/v1/jobs` can handle multiple generation modes via a discriminant `mode` field.

Routing Logic (`routes.py`):

- **Background Task Wrapper**: Uses FastAPI `BackgroundTasks` to immediately return a `job_id` and process the generation in the background.
- **In-Memory History**: Tracks job status (`QUEUED` , `GENERATING` , `COMPLETED` , `FAILED`) for polling.

---

4. Advanced Post-Processing: MeshOps Engine

Located in `hy3dgen/meshops/`, this is a significant extension over the base model.

Design Features:

- **DAG-based Execution**: Operations are topologically sorted based on a `depends_on` field, allowing complex pipelines (e.g., Cleanup → Decimate → Repair → Texture).
- **Blender Integration**: Uses a headless Blender pipeline for high-fidelity tasks like `.blend` conversion and native UV/Baking.
- **Native Baking**: High-to-low poly baking for Normal and AO maps.

---

5. IPC Bridge: `sidecarClient.js`

This file is the specific engineering solution for communicating between a (potentially) sandboxed frontend and the Python sidecar.

Essential Logic for Recreation:

1. **Dynamic Backend Detection**: Detects `window.location.origin` to route API calls correctly whether in Tauri or standard web browser.
2. **Request Normalization**: Wraps simple parameters into the complex `JobRequest` schema expected by the FastAPI backend.
3. **Smart Polling**: Implements an exponential/fixed-backoff polling mechanism to track background job completion.
4. **Parameter Unwrapping**: Handles IPC bridge nuances (like Tauri's `object` wrapping) to ensure data reaches Python in the correct Pydantic format.

---

6. Implementation Guide for AI Agents

To recreate these improvements from the base Hunyuan3D project, follow these steps:

1. **Wrap the Inference Pipeline**:

- Don't call the pipeline directly. Create an `InferencePipeline` class that encapsulates the model loading and `__call__` logic.

2. **Implement the Serial Manager**:
   - Create a singleton `PriorityRequestManager`.
   - Use `asyncio.run_in_executor` to offload the heavy Torch computation so it doesn't block the API event loop.
3. **Define JSON Schemas**:
   - Build Pydantic models for all inputs. Reject any request that doesn't strictly adhere to these schemas.
4. **Create the MeshOps Pipeline**:
   - Use `trimesh` for geometric modifications.
   - Implement a topological sort loop to process multiple operations on a single mesh instance.
5. **Build a State-less Frontend**:
   - Serve the HTML/JS from the FastAPI app itself using `app.mount("/", StaticFiles...)`.
   - Use `fetch` for all interactions to the `/v1` endpoints.
6. **Add System Telemetry**:
   - Add a `/v1/system/monitor` endpoint that returns GPU stats via `torch.cuda`.

---

7. Application Screenshots

7.1 Image-to-3D (Default View)

The primary interface tab where users upload an input image for 3D model generation. Includes settings for inference steps, seed, and texture generation toggle.

Image Tab

7.2 Multi-View Input

Four-view image input for multi-angle 3D reconstruction (Front, Back, Left, Right).

MultiView Tab

7.3 Text-to-3D Prompt

Text-based 3D generation with prompt and negative prompt fields for fine-grained control over the output.

Text Tab

7.4 Mesh Operations

Post-processing pipeline with Decimate, Repair, and Auto-Texture operations. Supports OBJ/GLB file inputs.

Mesh Tab

7.5 Export Modal

Model export dialog supporting GLB, OBJ, and STL output formats.

Export Modal